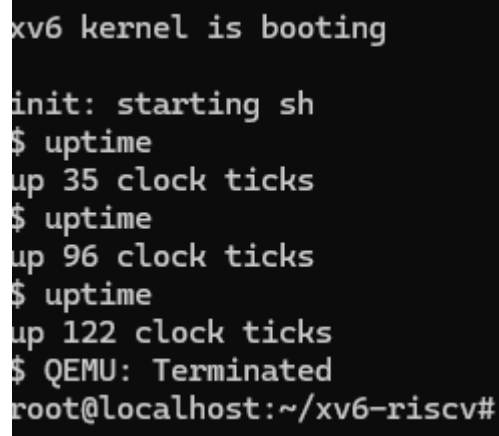


PA 1: Implementing xv6 User Commands and System Calls

Task 1. (5 pts) Implement the uptime utility

Uptime.c was created and pushed in the branch pa1-syscalls in the repository for PA1, it is in the user folder. This program calls uptime(), which is a system call that gets the number of clock ticks since the system runs, and prints that number.

Results:



```
xv6 kernel is booting
init: starting sh
$ uptime
up 35 clock ticks
$ uptime
up 96 clock ticks
$ uptime
up 122 clock ticks
$ QEMU: Terminated
root@localhost:~/xv6-riscv#
```

Task 2. (10 pts) Implement a time1 command that reports elapsed time.

time1.c is in the user directory, pushed into the pa1-syscalls branch, time1.c, sleep.c, matmul.c were added to UPGROS in Makefile.

If no command is given by the user, it prints a message letting the user know what happened and then it exits. The program gets the start ticks before the fork() call, when the call is made the child runs exec() command with its respective arguments. The parent calls wait(0) to wait for the child and then the parent stores the ending ticks with uptime(). The starting ticks are subtracted from the ending ticks and the result is printed.

```
xv6 kernel is booting

init: starting sh
$ time1 matmul
Time: 29 ticks
elapsed time: 29 ticks
$ time1 matmul & time1 matmul & time1 matmul &
$ Time: 81 ticks
elapsed time: 83 ticks
Time: 83 ticks
Time: 80 ticks
elapsed time: 82 ticks
elapsed time: 83 ticks
time1 sleep 10
elapsed time: 10 ticks
$ |
```

Task 3. (10 pts) Keep track of how much cputime a process has used.

The files kernel/proc.h, kernel/proc.c, and kernel/trap.c were modified.

-proc.h: uint64 cputime; line was added at the end of struct proc to give every process their own ticks which are CPU time counters.

-proc.c: the line p->cputime=0; was added in the static struct allocproc() after p->state = USED; line. This line makes new processes begin at 0, so it does not use the value that a previous process had.

-trap.c: inside usertrap(void) in the condition if(which_dev==2){yield();} the line p->cputime++; was added. This condition is used when the process spent all its time slice while running the user's code, with this new line a tick is added to the process counter before it lets another process use the CPU.

Another line was added inside kerneltrap(). In the condition if(which_dev==2 && myproc() != 0){yield();} the line myproc()->cputime++; is written before yield(). This addition charges a tick to the process counter when the time goes off while running kernel code.

All changes were committed and pushed.

Task 4. (10 pts) Implement a wait2() system call that waits for a child to exit and returns the child's status and rusage.

The files kernel/pstat.h, user/user.h, user/usys.pl, kernel/syscall.h, kernel/syscall.c, kernel/sysproc.c, kernel/defs.h, kernel/proc.c were modified.

-kernel/pstat.h: this file was created, and inside it defines the struct for resource usage.

-user/user.h: the lines struct rusage; and int wait2(int, struct rusage); were added to let the prototype use struct rusage*, so the prototype lets user programs call wait2().

- user/usys.pl: entry("wait2") was added to generate the assembly stub in usys.S
- kernel/syscall.h: #define SYS_wait2 23 was added to give the new system call a number that is unique
- kernel/syscall.c: the lines extern uint64 sys_wait2(void); and [SYS_wait2] sys_wait2 were written in syscalls[] table to add the system call information of wait2().
- kernel/sysproc.c: argaddr(0,),argaddr(1,) and kwait2(p,r) were included in sys_wait2() which was also added. All of this to get the system call return child process status and rusage, were rusage will only be cputime.
- kernel/defs.h: int kwait2(uint64,uint64); was added after the kwait prototype to make kwait2() visible to sysproc.c
- kernel/proc.c: #include "pstat.h" and kwait2() were implemented so the kernel can use struct rusage. Kwait2() was based almos entirely on kwait(), it has a second address argument added.

All changes and creations were committed and pushed into de repository.

Task 5. (10 pts) Implement a time command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.

time.c was created and based on time1.c, it was also added to UPROGS, it calls wait2() only, it computes the percentage of CPU, and includes kernel/pstat.h to use struct rusage.

```
xv6 kernel is booting

init: starting sh
$ matmul
Time: 28 ticks
$ time matmul
Time: 28 ticks
elapsed time: 28 ticks, cpu time: 28 ticks, 100% CPU
$ time matmul & time matmul &
$ Time: 55 ticks
elapsed time: 56 ticks, cpu time: 28 ticks, 50% CPU
Time: 54 ticks
elapsed time: 56 ticks, cpu time: 28 ticks, 50% CPU
time matmul 4
Time: 113 ticks
elapsed time: 113 ticks, cpu time: 113 ticks, 100% CPU
$ time sleep 10
elapsed time: 10 ticks, cpu time: 0 ticks, 0% CPU
$ time matmul 5
Time: 141 ticks
elapsed time: 141 ticks, cpu time: 141 ticks, 100% CPU
$ |
```

Task 6. (5 pts) Summarize what you learned from doing this assignment. Describe any difficulties you faced and if/how you overcame them.

One of the main things that I learnt with this programming assignment was the immense connection that different components do to make the system work. Fork(), exec(), and wait collaborate to run and time another program. Also, I was able to see how timer interrupts are responsible of managing the turns on the CPU. Even though process take their own time, the system switching between tasks make it look as if each process were all running at the same time with the same resources.

I faced some difficulties while doing this activity. Getting everything to work in Github inside a virtual machine that is not in my personal computer was challenging. Also, getting familiar with many new commands, many of which I had to become familiar with again since I have used them before in other projects. Additionally, debugging the code that I wrote and ordering what I wanted to do was a challenge as well, since it is different to work from linux than from the usual for me which is Windows. And the biggest challenge was understanding how everything works, how the system makes sense, how the code that I am writing in the kernel/ impacts in user/, it took me several days to start having a notion of what was happening and why.

AI use note: I used Claude to create a guide for explanations for this assignment to help fixing Git problems as well as for debugging and directing me in the right direction to make the programs work, particularly on the kwait2() in task 4 where I struggled to modify and implement the part that is responsible to handle a ZOMBIE child. I wrote, built, and tested the code myself on my Linode VM.